FPGA / ARTY A7 · COURSE 7

The Servo Guard

The reflex guard, now clamping a real servo's travel — switches set a demand angle, the fabric clamps it and drives the servo. The clamp you can feel, in logic.

ARTY-007 · ~45 MIN · INTERMEDIATE

1. The one idea

The same guard contract you've run on an LED and a thermometer — `safe = min(demand, limit)` in Q8 — now drives a **real servo**. The slide switches ask for an angle; the fabric clamps it to a safe ceiling and generates the servo pulse for the *clamped* angle; the arm stops at the limit while you keep raising the switches.

2. Why it matters

Every Arty course so far clamped an LED bar or a number. This one clamps **motion** — the guard's first real actuator. And it shows the FPGA's robotics edge: a servo pulse generated in logic is **jitter-free** and scales to many servos at once with no added timing slop. That deterministic, parallel real-time control is exactly what a CPU loop can't promise — the reason motion control belongs in fabric.

3. Parts & wiring

- Arty A7 (onboard SW3..SW0, LD4..LD7)
- SG90 micro servo
- Pmod 8LD (optional — echoes the clamp as a thermometer)
- Bench PSU at 5V for the servo

Signal	From	To	Why
Demand	SW3..SW0	the fabric	`demand = sw * 17` (0..255)
Servo pulse	a Pmod pin (e.g. JA1)	servo signal (orange)	the 3.3V pulse is a valid servo signal
Servo power	bench 5V	servo +(red)	servos spike current — **never** from the Arty
Servo ground	bench GND	servo −(brown) **and** Arty GND	a common ground, or the pulse is meaningless

> *Two FPGA habits and one bench habit.* ***Synchronize the switches*** *with a two-flop synchronizer before the logic uses them — they're asynchronous to the clock.* ***Constrain the servo output pin*** *in the* `.xdc` *; an unconstrained I/O is a silent timing bug. And on the bench: external 5V for the servo, common ground — the same rule the ESP32 servo lesson taught.*

4. Predict (do this before uploading)

- At full switches (demand 255 → 180°) with the default 0.60 limit, where does the arm actually stop?
- What pulse width (ms) does that clamped angle produce?
- Raise the switches past the limit — does the pulse keep widening, or hold?

5. Build it — precise timing, then the clamp

The servo pulse is a frame counter in fabric — a 20 ms frame, a high-time of 1.0–2.0 ms set by the angle:

```
 // servo_pwm.sv — 50 Hz frame, angle_q8 (0..255) → 1.0..2.0 ms pulse
localparam integer PER_US    = CLK_HZ / 1_000_000;          // 100 clocks / us
localparam integer FRAME_CNT = PER_US * 20_000;             // 20 ms frame
localparam integer MIN_CNT   = PER_US * 1000;               // 1.0 ms at angle 0
localparam integer SPAN_CNT  = PER_US * 1000;               // +1.0 ms span to full

wire [31:0] high_cnt = MIN_CNT + (angle_q8 * SPAN_CNT) / 255;

always @(posedge clk) begin
  frame ≤ (frame == FRAME_CNT - 1) ? '0 : frame + 1'b1;
  servo ≤ (frame < high_cnt);      // high for the first high_cnt clocks each frame
end
```

And the guard — the same one line, now feeding the servo:

```
 // servo_guard_top.sv
wire [7:0] demand_q8 = (sw * 8'd17 > 255) ? 8'd255 : (sw * 8'd17);
wire [7:0] safe_q8   = (demand_q8 > LIMIT) ? LIMIT : demand_q8;   // the guard

servo_pwm u_servo (.clk(CLK100MHZ), .angle_q8(safe_q8), .servo(servo_pwm));
```

The new HDL skill is **generating precise timing in fabric** — a counter that makes a clean pulse every clock, no jitter. The guard is the familiar clamp; the discipline is driving the servo from `safe_q8`, never `demand_q8`.

6. Test against your prediction

Test in **simulation first** — the golden vectors lock the math: angle 0 → 1.0 ms (100 000 clocks), full scale → 2.0 ms (200 000 clocks), and a demand of 255 clamped to 153, so the pulse reflects ~108°, not 180°. Run them with `npx vitest run servoGuardModel`. Then, on the board (with approval): raise the switches and the arm follows, then **freezes at the limit** while you keep raising them — and the 8LD bar caps in lockstep. The clamp, in motion.

7. What goes wrong (pre-loaded debugging)

- **Servo won't move / the Arty browns out** → you powered the servo from the board. External 5V, common ground.
- **Arm twitches or ignores the demand** → no common ground between the 5V supply and the Arty.
- **No clamp — the arm reaches full travel** → you drove `demand_q8` into the PWM instead of `safe_q8`. The same bug as the ESP32 servo guard, in HDL.
- **The angle is wrong everywhere** → the `CLK_HZ` constant doesn't match the real clock, so the counts are off. The pulse timing is only as right as that one number.
- **The arm jitters** → the pulse is gated through slow or unconstrained logic; keep the PWM register on the system clock and constrain the output pin.

8. Checkpoint

The guard you met on an LED now clamps a real joint — in pure logic, jitter-free, and the clamp is *physical*. Same contract, real motion, deterministic timing. This is the Arty doing what it's best at for robotics: precise, parallel, real-time control with a safety clamp baked into the fabric.

9. Push further

- Drive **several servos at once** from the same fabric — the FPGA's real edge, with zero added jitter.
- Feed the demand from a **Pmod ENC** rotary encoder instead of the switches.
- Add a **slew-rate limit** in logic so the arm can't snap — the `maxStep` field of the same kernel.
- Stream demand / safe / pulse over UART to the live dashboard — the guard's decision, recorded.